

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

SIOPTR: TT6: Week 3

**My Own KERnel (MOKER) version: Framework Tracing
Mechanism**

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 Introduction

The goal of this feature is to create scheduling tracing mechanism in the Linux kernel. The purpose is to register the event types: `SCHED_TICK`, `SWITCH_TO`, `SWITCH_AWAY`, and also `MISC_EVT`.

The `SCHED_TICK` event happens whenever the periodic interrupt, called `tick`, is fired. Note that, there is always a task executing when such event occurs.

The `SWITCH_TO` event happens whenever a task is assigned to CPU for execution, while `SWITCH_AWAY` event happens whenever a task is removed from CPU.

The `MISC_EVT` is used for unspecified events. This feature is a module, because it is created a new entry in the `proc` directory to pass the scheduling information from kernel to user space. So, the interaction is through a `proc` entry. So, it creates the `proc/moker_trace` entry.

To store the scheduling data, this module implements a ring buffer. It saves the the following information:

- Event type
- Event timestamp,
- Trace number (this field could be used to differentiate two equal "simultaneous" events)
- Process policy
- Process priority
- Process identifier
- Process state
- Process name.

2 Configuration Option Entry

It must be created the `MOKER_TRACING` configuration entry that depends on `MOKER_FRAMEWORK` entry.

Open the `linux-6.19.9-moker/kernel/moker/Kconfig` file and update it by adding (blue text):

```

menu "MOKER framework"
config MOKER_FRAMEWORK
bool "My Own Kernel Framework"
default y
config MOKER_TRACING
bool "MOKER tracing"
default y
depends on MOKER_FRAMEWORK
endmenu

```

3 Coding

Two source code files are created into the `linux-6.19.9-moker/kernel/moker` directory: `trace.h` and `trace.c`.

The content of `trace.h` is:

```

#ifndef __TRACE_H_
#define __TRACE_H_

#define TRACE_ENTRY_NAME "moker_trace"
#define TRACE_BUFFER_SIZE 1000
#define TRACE_STRING_BUFFER_SIZE 200
#define TRACE_TASK_COMM_LEN 16

enum evt{
    MISC_EVT = 0,
    SCHED_TICK,
    SWITCH_AWAY,
    SWITCH_TO,
};

struct trace_evt{
    enum evt event;
    unsigned long long time;
    int number;
    pid_t pid;
    int state;
    int prio;
    int policy;
    char comm[TRACE_TASK_COMM_LEN];
};

struct trace_evt_buffer{
    struct trace_evt events[TRACE_BUFFER_SIZE];
    int write_item;
    int read_item;
    spinlock_t lock;
};

ssize_t trace_read(struct file *filp, char __user *buf, size_t count, loff_t *f_pos);
void moker_trace(enum evt event, int number, struct task_struct *p);
#endif

```

The content of `trace.c` is:

```

#include <linux/module.h>

```

```

#include <linux/kernel.h>
#include <linux/proc_fs.h>
#include <linux/sched.h>
#include <asm/uaccess.h>
#include <linux/uaccess.h>

#include "trace.h"

struct trace_evt_buffer trace;

static void increment(int * item){
    *item = *item + 1;
    if(*item >= TRACE_BUFFER_SIZE){
        *item = 0;
    }
}

static int is_empty(int r, int w){
    //empty if r == w
    //otherwise r != w
    return !(r ^ w); //xor
}

static int is_full(int r, int w){
    int write = w;
    increment(&write);
    return write == r;
}

static int dequeue (char *buffer){
    int ret = 0, len;
    char evt[10];
    spin_lock(&trace.lock);
    if(!is_empty(trace.read_item, trace.write_item)){ //if it is not empty

        switch((int)trace.events[trace.read_item].event){
            case MISC_EVT:
                strcpy(evt, "MC_EVT");
                break;
            case SCHED_TICK:
                strcpy(evt, "SCH_TK");
                break;
            case SWITCH_AWAY:
                strcpy(evt, "SWT_AY");
                break;
            case SWITCH_TO:
                strcpy(evt, "SWT_TO");
                break;
            default:
                strcpy(evt, "UK_EVT");
        }

        len = sprintf(buffer, "%llu", trace.events[trace.read_item].time);
        len += sprintf(buffer + len, "%s", evt);
        len += sprintf(buffer+len, "%d", (int)trace.events[trace.read_item].number);
        len += sprintf(buffer+len, "%d", (int)trace.events[trace.read_item].policy);
        len += sprintf(buffer+len, "%d", (int)trace.events[trace.read_item].prio);
        len += sprintf(buffer+len, "%d", (int)trace.events[trace.read_item].pid);
        len += sprintf(buffer+len, "%d", (int)trace.events[trace.read_item].state);
        len += sprintf(buffer+len, "%s\n", trace.events[trace.read_item].comm);

        increment(&trace.read_item);
        ret = 1;
    }
    spin_unlock(&trace.lock);
    return ret;
}

static int enqueue (enum evt event, unsigned long long time, int number, struct task_struct *p)
{
    spin_lock(&trace.lock);
    if(is_full(trace.read_item, trace.write_item))
        increment(&trace.read_item);
    trace.events[trace.write_item].number = number;
    trace.events[trace.write_item].event = event;
    trace.events[trace.write_item].time = time;
}

```

```

    trace.events[trace.write_item].pid = p->pid;
    trace.events[trace.write_item].state = p->__state;
    trace.events[trace.write_item].prio = p->prio;
    trace.events[trace.write_item].policy = p->policy;
    strcpy(trace.events[trace.write_item].comm, p->comm);

    increment(&trace.write_item);
    spin_unlock(&trace.lock);
    return 1;
}

ssize_t trace_read(struct file *filp, char __user *buf, size_t count, loff_t *f_pos)
{
    char buffer[TRACE_STRING_BUFFER_SIZE];
    int ret = 0, len = 0;

    if(!dequeue(buffer))
        return 0;
    len = strlen(buffer);
    if(len <= 0)
        return -EFAULT;
    if(count < len)
        return -EFAULT;
    ret=copy_to_user(buf,buffer,len);
    if(ret != 0)
        return -EFAULT;
    return len;
}

static const struct proc_ops trace_ops = {
    .proc_read = trace_read,
};

static int __init proc_trace_init(void){
    proc_create(TRACE_ENTRY_NAME,0444, NULL, &trace_ops);
    printk("MOKER:/proc/%s created\n", TRACE_ENTRY_NAME);
    spin_lock_init(&trace.lock);
    trace.write_item = 0;
    trace.read_item = 0;
    return 0;
}

module_init(proc_trace_init);

void moker_trace(enum evt event, int number, struct task_struct *p){
    unsigned long long time;

    time = ktime_to_ns(ktime_get());
    enqueue(event,time, number, p);
}

```

4 Putting the MOKER Trace code Available

In order to use the trace code, first it needs to be known. For that purpose, we have to include it into the Linux kernel scheduling framework code.

Open the `linux-6.19.9-moker/kernel/sched/sched.h` file and update it by adding:

```

...

struct rq;
struct cfs_rq;
struct rt_rq;
struct sched_group;
struct cpuidle_state;

#ifdef CONFIG_PARAVIRT
# include <asm/paravirt.h>
# include <asm/paravirt_api_clock.h>
#endif

#include <asm/barrier.h>

#include "cpupri.h"
#include "cpudeadline.h"

#ifdef CONFIG_MOKER_TRACING
#include "../moker/trace.h"
#endif

/* task_struct::on_rq states: */
#define TASK_ON_RQ_QUEUED 1
#define TASK_ON_RQ_MIGRATING 2
...

```

5 Collecting data

Now, let collect events. Therefore, we have to invoke `MOKER trace` function in two places.

First, to collect `SCHED_TICK` events, open the `linux-6.19.9-moker/kernel/sched/core.c` file and update it by adding to the `sched_tick` function:

```

/*
 * This function gets called by the timer code, with HZ frequency.
 * We call it with interrupts disabled.
 */
void sched_tick(void)
{
    int cpu = smp_processor_id();
    struct rq *rq = cpu_rq(cpu);
    /* accounting goes to the donor task */
    struct task_struct *donor;
    struct rq_flags rf;
    unsigned long hw_pressure;
    u64 resched_latency;

    if (housekeeping_cpu(cpu, HK_TYPE_KERNEL_NOISE))
        arch_scale_freq_tick();

    sched_clock_tick();

    rq_lock(rq, &rf);
    donor = rq->donor;

#ifdef CONFIG_MOKER_TRACING
    moker_trace(SCHED_TICK, -1, donor);
#endif

    psi_account_irqtime(rq, donor, NULL);

    update_rq_clock(rq);
    hw_pressure = arch_scale_hw_pressure(cpu_of(rq));
    update_hw_load_avg(rq_clock_task(rq), rq, hw_pressure);

    if (dynamic_preempt_lazy() && tif_test_bit(TIF_NEED_RESCHED_LAZY))
        resched_curr(rq);

    donor->sched_class->task_tick(rq, donor, 0);

    if (sched_feat(LATENCY_WARN))
        resched_latency = cpu_resched_latency(rq);
    calc_global_load_tick(rq);
    sched_core_tick(rq);
    scx_tick(rq);

    rq_unlock(rq, &rf);

    if (sched_feat(LATENCY_WARN) && resched_latency)
        resched_latency_warn(cpu, resched_latency);

    perf_event_task_tick();

    if (donor->flags & PF_WQ_WORKER)
        wq_worker_tick(donor);

    if (!scx_switched_all()) {
        rq->idle_balance = idle_cpu(cpu);
        sched_balance_trigger(rq);
    }
}

```

Second, to collect SWITCH_AWAY and SWITCH_TO events, open the `linux-6.19.9-moker/kernel/sched/core.c` file and update it by adding to the `__schedule` function:

```

static void __sched notrace __schedule(int sched_mode)
{
    struct task_struct *prev, *next;
    /*
     * On PREEMPT_RT kernel, SM_RTLOCK_WAIT is noted
     * as a preemption by schedule_debug() and RCU.
     */
    bool preempt = sched_mode > SM_NONE;
    bool is_switch = false;
    unsigned long *switch_count;
    unsigned long prev_state;
    struct rq_flags rf;
    struct rq *rq;
    int cpu;

    /* Trace preemptions consistently with task switches */
    trace_sched_entry_tp(sched_mode == SM_PREEMPT);

    cpu = smp_processor_id();
    rq = cpu_rq(cpu);
    prev = rq->curr;

    ...

    pick_again:
    next = pick_next_task(rq, rq->donor, &rf);
    rq_set_donor(rq, next);
    if (unlikely(task_is_blocked(next))) {
        next = find_proxy_task(rq, next, &rf);
        if (!next)
            goto pick_again;
        if (next == rq->idle)
            goto keep_resched;
    }
    picked:
    clear_tsk_need_resched(prev);
    clear_preempt_need_resched();
    keep_resched:
    rq->last_seen_need_resched_ns = 0;

    is_switch = prev != next;
    if (likely(is_switch)) {
        rq->nr_switches++;
        ...
        ++switch_count;

        psi_account_irqtime(rq, prev, next);
        psi_sched_switch(prev, next, !task_on_rq_queued(prev) ||
            prev->se.sched_delayed);

        trace_sched_switch(preempt, prev, next, prev_state);
    }

#ifdef CONFIG_MOKER_TRACING
    moker_trace(SWITCH_AWAY, -1, prev);
    moker_trace(SWITCH_TO, -1, next);
#endif

    /* Also unlocks the rq: */
    rq = context_switch(rq, prev, next, &rf);
} else {
    /* In case next was already curr but just got blocked.donor */
    if (!task_current_donor(rq, next))
        proxy_tag_curr(rq, next);

    rq_unpin_lock(rq, &rf);
    __balance_callbacks(rq, NULL);
    raw_spin_rq_unlock_irq(rq);
}
    trace_sched_exit_tp(is_switch);
}

```

6 Compiling

In order to compile to include the trace code in the compiling process, you have to update the `Makefile` file in the `linux-6.19.9-moker/kernel/moker` directory.

```
# MOKER framework makecatfile
obj-$(CONFIG_MOKER_TRACING) += trace.o
```

Then, before compiling update the `.config` file.

1. For that purpose, open a terminal `$HOME/linux-kernel` directory:
and type:
`cd linux-6.19.9-moker`
2. Check if the `MOKER_TRACING` configuration option is selected for compilation:
`make menuconfig`
Select "Exit" and "Yes" for saving the new `.config` file.
`cd ..`
3. Compile and install the kernel:
`sudo ./kcompile.sh`
`sudo reboot`

7 Getting data

After rebooting and logging in, open a terminal for getting the scheduling trace:

```
cat /proc/moker_trace > trace.csv
cat trace.csv
```

```

152498326697,SWT_AY,*-1*,0,120,2461,0,gnome-shell
152498326798,SWT_TO,*-1*,0,120,2288,0,Xorg
152498434673,SCH_TK,*-1*,0,120,2288,0,Xorg
152498734704,SWT_AY,*-1*,0,120,2288,1,Xorg
152498734839,SWT_TO,*-1*,0,120,2461,0,gnome-shell
152504317957,SWT_AY,*-1*,0,120,2461,0,gnome-shell
152504318072,SWT_TO,*-1*,0,120,2288,0,Xorg
152504376574,SCH_TK,*-1*,0,120,2288,0,Xorg
152505513349,SWT_AY,*-1*,0,120,2288,1,Xorg
152505513423,SWT_TO,*-1*,0,120,2461,0,gnome-shell
152505535263,SWT_AY,*-1*,0,120,2461,0,gnome-shell
152505535324,SWT_TO,*-1*,0,120,2288,0,Xorg
152505544026,SWT_AY,*-1*,0,120,2288,1,Xorg
152505544075,SWT_TO,*-1*,0,120,2786,0,gnome-terminal-
152505570302,SWT_AY,*-1*,0,120,2786,1,gnome-terminal-
152505570374,SWT_TO,*-1*,0,120,2461,0,gnome-shell
152505595911,SWT_AY,*-1*,0,120,2461,0,gnome-shell
152505595977,SWT_TO,*-1*,0,120,2288,0,Xorg
152505603093,SWT_AY,*-1*,0,120,2288,1,Xorg
152505603153,SWT_TO,*-1*,0,120,2786,0,gnome-terminal-
152505615037,SWT_AY,*-1*,0,120,2786,1,gnome-terminal-
152505615101,SWT_TO,*-1*,0,120,2461,0,gnome-shell
152505628239,SWT_AY,*-1*,0,120,2461,1,gnome-shell
152505628304,SWT_TO,*-1*,0,120,0,0,swapper/0
152507009004,SCH_TK,*-1*,0,120,0,0,swapper/0
152515618248,SWT_AY,*-1*,0,120,0,0,swapper/0

```

This tracing mechanism outputs a set of data separated by comma. Each line is formatted as:

<timestamp>, <event>, <*number*>, <policy>, <prio>, <pid>, <state>, <comm>

The first column refers to the timestamp and second one to the event type. The following columns present task's (or process) data:

- Scheduling policy (policy), defined into linux-6.19.9-moker/include/uapi/linux/sched.h.
 - The policy range of values are:

```

/*
 * Scheduling policies
 */
#define SCHED_NORMAL 0
#define SCHED_FIFO 1
#define SCHED_RR 2
#define SCHED_BATCH 3
/* SCHED_ISO: reserved but not implemented yet */
#define SCHED_IDLE 5
#define SCHED_DEADLINE 6
#define SCHED_EXT 7

```

- Priority level (prio)
 - The prio value varies from 0 to 139
- The process identifier(pid)
 - The pid value varies from 0 to approximately 4 millions (2^{22})

- Process state (`state`), defined into `linux-6.19.9-moker/include/linux/sched.h`.
 - The `state` range of values are:

```
/* Used in task->state: */
#define TASK_RUNNING 0x00000000
#define TASK_INTERRUPTIBLE 0x00000001
#define TASK_UNINTERRUPTIBLE 0x00000002
#define __TASK_STOPPED 0x00000004
#define __TASK_TRACED 0x00000008
/* Used in task->exit_state: */
#define EXIT_DEAD 0x00000010
#define EXIT_ZOMBIE 0x00000020
#define EXIT_TRACE (EXIT_ZOMBIE | EXIT_DEAD)
/* Used in task->state again: */
#define TASK_PARKED 0x00000040
#define TASK_DEAD 0x00000080
#define TASK_WAKEKILL 0x00000100
#define TASK_WAKING 0x00000200
#define TASK_NOLOAD 0x00000400
#define TASK_NEW 0x00000800
#define TASK_RTLOCK_WAIT 0x00001000
#define TASK_FREEZABLE 0x00002000
#define __TASK_FREEZABLE_UNSAFE (0x00004000 * IS_ENABLED(CONFIG_LOCKDEP))
#define TASK_FROZEN 0x00008000
#define TASK_STATE_MAX 0x00010000

#define TASK_ANY (TASK_STATE_MAX-1)
```

- Process name (`comm`)
 - The process executable name